

# Athenaeum — The Body

- [Design Document: Athenaeum — The Body](#)
  - [1. Purpose](#)
  - [2. Hardware & Infrastructure Inventory](#)
  - [3. Storage Tier Architecture](#)
  - [4. Compute Topology](#)
  - [5. Resident State Model \(structural, not cognitive\)](#)
  - [6. Elastic Resource Management](#)
  - [7. Concurrency Execution Mechanics \(structural only\)](#)
  - [8. Configuration Surface \(Body-relevant subset\)](#)
  - [9. Ingestion Pipeline \(mechanical only\)](#)
  - [10. Resilience & Failure Modes Summary](#)
  - [11. Work Breakdown \(Body-only phases, from the full task backlog\)](#)

## Design Document: Athenaeum — The Body

**Status:** Scoped extract for active development (rev. 3 — adds the Local Model Serving Layer and a sandboxed execution capability to support strong coding competence; rev. 2 cherry-picked content-addressed integrity, idempotent/versioned writes, and recovery-playbook structure from the alternate v1.0 review package; the elastic Proxmox infrastructure model itself is unchanged throughout) **Scope:** Infrastructure, storage, resource elasticity, concurrency execution mechanics, ingestion mechanics, and configuration. This document deliberately excludes all cognitive logic. **Explicitly out of scope (see the full system design document for these):** - The Master Agents and the deliberation loop’s reasoning content (framing, exploration, cross-examination, synthesis) - The Reputability Engine’s judgment logic (grading rules, dispute adjudication, standard evolution) - Re-evaluation *judgment* (deciding whether a Belief Graph change is “material enough” to warrant re-answering)

Everywhere those would normally plug in, this document defines only the **interface/contract** they must satisfy — a stub or no-op implementation is sufficient to build and test everything here. The Body must be complete, testable, and stable on its own, before the Brain is built against it.

**Provenance note:** an alternate design package proposed replacing this Body entirely with a single-VM, non-elastic, specific-tech-stack infrastructure. That replacement is declined — the elastic Proxmox/tiered-storage model below remains the design. Two of its concrete engineering mechanisms are adopted regardless, because they strengthen this model rather than replace it: content-addressed, tamper-evident storage (Section 3.4) and idempotent, version-checked writes as the concurrency-control mechanism Section 7.3 previously left unspecified (Section 7.5).

---

### 1. Purpose

The Body is everything that lets a “mind” exist somewhere: it boots, it remembers, it survives resource loss, it holds a queue of work, it moves data in and out, and it hands well-formed questions to whatever reasoning component is plugged in — without needing to know or care what that component actually does. Its correctness is measured by durability, graceful degradation, and interface stability, not by the quality of any answer.

---

### 2. Hardware & Infrastructure Inventory

| Resource                       | Baseline (guaranteed)                     | Elastic ceiling (opportunistic) | Notes                                      |
|--------------------------------|-------------------------------------------|---------------------------------|--------------------------------------------|
| Compute (local)                | 1 core                                    | 40 cores                        | Proxmox VM/LXC on local hypervisor         |
| Compute (distributed)          | 0 cores                                   | ~200 cores                      | Remote pool, network-dependent, may vanish |
| Memory (local, DDR3)           | 512 GiB (must be guaranteed at all times) | 512 GiB fixed                   | The one resource treated as non-negotiable |
| Memory (distributed DRAM pool) | 0                                         | ~1 TiB                          | Overflow/cache only, never authoritative   |
| GPU                            | 0                                         | 10 GPUs / 240 GB VRAM combined  | Opportunistic acceleration only, never     |

|                            |                       |                                    |                                                    |
|----------------------------|-----------------------|------------------------------------|----------------------------------------------------|
| Local bulk storage         | 500 TiB spinning HDD  | fixed                              | load-bearing<br>Cold archive, corpora, checkpoints |
| Local boot storage         | SSD array             | fixed                              | OS + database image, loaded at boot                |
| Network high-speed storage | 50 TiB NVMe (via LAN) | fixed, availability not guaranteed | Hot working set, indices                           |
| Virtualization layer       | Proxmox VE            | —                                  | Host for primary guest and orchestrated workers    |

**Design consequence:** 512 GiB local DRAM is the only hard dependency. Every other resource is an optional accelerator that can appear or disappear without corrupting or losing state.

---

## 3. Storage Tier Architecture

|                                       |                                                             |
|---------------------------------------|-------------------------------------------------------------|
| Tier 0 – DRAM (local, 512 GiB)        | : live working memory, resident state                       |
| Tier 1 – Boot SSD array               | : OS image, database engine, hot config                     |
| Tier 2 – NVMe over LAN (50 TiB)       | : hot corpus shards, indices, recent checkpoints            |
| Tier 3 – Local spinning HDD (500 TiB) | : cold archive – full corpus, full checkpoint history, logs |

### 3.1 Boot behavior

- On boot, a minimal OS + the persistence/database engine loads from the local SSD array (Tier 1), treated purely as a bootstrap volume, not long-term knowledge storage.
- After boot, the database engine reconstructs the most recent resident state by streaming the latest checkpoint from Tier 2 (preferred) or Tier 3 (fallback) into DRAM.
- Boot is a “resume from hibernation” by default; a `bootstrap_mode: genesis` flag forces a cold start with empty state (see Section 8).

### 3.2 Tier roles

- **Tier 0:** the only tier where active processing happens.
- **Tier 1:** system image; emergency local checkpoint target if Tier 2 is unreachable and Tier 3 is saturated.
- **Tier 2:** primary read/write cache and primary checkpoint destination during normal operation.
- **Tier 3:** system of record — full corpus and full append-only checkpoint history. Never on the critical path for a single processing step, only for durability.

### 3.3 Availability assumptions

- Tier 2 is not guaranteed available. The Body must detect its absence, fall back to Tier 3, and automatically resume preferring Tier 2 when it reappears — no manual fallback.
- Tier 0 is assumed always available for the life of a running session; if it drops below the working-set floor, the Body checkpoints and suspends rather than continuing (Section 6).

### 3.4 Content-addressed, tamper-evident storage

Every object written to Tier 2/3 by the persistence layer (checkpoints, Provenance Ledger blobs, corpus shards) is stored **content-addressed**: its storage key is derived from a hash of its own content, not an arbitrary ID. Two consequences follow directly, and both are structural guarantees the Body provides — what they’re *used for* is a Brain concern (e.g., Reputability grading treating a source’s identity as tied to its exact content, `brain-design.md` Section 6.2):

- **Tamper evidence:** any modification to stored content changes its address, so a checkpoint or source blob silently altered on disk is immediately detectable on read (address/content mismatch), rather than silently served as if unchanged.
- **Deduplication and stable references:** identical content referenced from multiple places (e.g., a source cited by many claims, or an unchanged corpus shard across several checkpoints) is stored once, and every reference to it is a durable hash rather than a path that could later point somewhere else.

- Each entry in the append-only checkpoint log (Section 5.3) additionally carries a reference to the hash of the immediately preceding entry, forming a hash chain — so the log’s own integrity (nothing inserted, removed, or reordered) is verifiable without trusting the storage medium itself, the same durability guarantee `body-design.md` already makes, made checkable rather than merely asserted.

---

## 4. Compute Topology

### 4.1 Local compute (Proxmox, up to 40 cores)

Handles orchestration, checkpointing/serialization, corpus ingestion mechanics, and whatever workload is handed to it by the (external, stubbed) reasoning component. The Body has no opinion on what runs on these cores beyond scheduling it fairly and durably.

## 4.2 Distributed compute (opportunistic, up to ~200 cores)

Reserved for embarrassingly parallel, checkpoint-safe work units dispatched by the scheduler: batch re-indexing, batch verification jobs, or parallel execution of independent queue items. Workers are **stateless lease-holders** — they receive a work unit and minimal context, return a result, and hold no unique state. Loss of a worker mid-task simply requeues the unit.

## 4.3 GPU pool (opportunistic, up to 10 GPUs / 240 GB VRAM)

Accelerators only, never dependencies. Every GPU-accelerated code path (embedding generation, similarity search) must have a CPU-only fallback. The Body must never block on GPU availability.

## 4.4 The single-core, full-memory floor

This is the Body's stated minimum viable operating point: - All scheduled work must be expressible as cooperatively-scheduled, single-thread-capable units — correctness may never depend on true parallelism, only speed does. - The execution model (Section 7) is a serializable sequence of discrete steps, each executable one at a time on one core without restart. - Throughput scales down as compute shrinks toward this floor; this is accepted by design.

## 4.5 Local Model Serving Layer

The Brain's Master Agents don't reason in the abstract — each round handler (Section 1.2 of `brain-design.md`) is ultimately backed by inference against a local language model. The Body is responsible for making any number of heterogeneous local models (general-purpose, coding-specialized, math-specialized, etc.) available on demand, loaded and unloaded elastically against the GPU/VRAM pool (4.3), with the same CPU-fallback discipline as everything else GPU-related. This is infrastructure, not judgment: *which* model is best suited to a given agent and task is Brain-owned Model Fitness judgment (`brain-design.md` Section 6.7); *making the requested model available, quickly and reliably, within current resource constraints* is this layer's job.

**4.5.1 Model registry.** A catalog of admitted models, each entry holding: model identifier, parameter count and quantization, VRAM footprint at that quantization, required backend, and content-addressed storage location for the weight files (Section 3.4 — a corrupted or tampered model file is caught by the same hash-mismatch-on-read mechanism as any other stored object). Admission to the registry is a Brain-owned decision (Model Fitness, 6.7); this layer only stores and serves what's admitted.

**4.5.2 Serving architecture: a thin router over two backends, not one monolithic tool.** - **GPU-backed serving** for concurrent, throughput-sensitive workloads: an engine built for continuous batching and multi-request concurrency across the GPU pool (4.3), so that several Master Agents' round handlers drawing on models at once don't serialize behind each other unnecessarily. - **CPU-backed fallback serving** for the degraded-resource and single-core-floor cases (4.4): a lightweight, dependency-minimal inference engine that runs without GPU access at all, satisfying the same "every GPU path has a CPU fallback" rule already established for embedding/retrieval (Section 9, tasks 21–22). - A **unified routing interface** in front of both: round handlers request a model by name or by required capability (e.g., "a coding-capable model") and receive a response through one consistent contract, never addressing a specific backend or GPU directly. This is what lets the Brain's claim structure record a `serving_model` (`brain-design.md` Section 3.5) without needing to know or care which backend actually served it. - A GUI-based, human-facing model workbench (evaluating and comparing candidate models before admission) is a reasonable and useful tool at this stage, but is explicitly **not** the production serving path — it's a pre-admission evaluation step feeding the registry (4.5.1), not something the running system depends on at query time. Conflating the two (relying on a single-user desktop tool as the unattended, elastic, multi-agent serving backend) is the specific mistake this design avoids.

**4.5.3 Elastic loading and VRAM budget management.** The combined VRAM pool (4.3) is a shared, contended resource across every concurrently-loaded model, managed with the same discipline as Section 5.4's memory pressure handling, applied to VRAM specifically: 1. Models not currently in active use are unloaded first (least-recently-used), freeing VRAM for a newly requested model — this is a routine, expected operation, not a failure. 2. If VRAM remains insufficient even after unloading idle models, a request for a GPU-backed model is either queued (if the caller can tolerate waiting) or transparently redirected to the CPU-backed fallback path (4.5.2) — the caller gets a working model, at reduced speed, rather than an error. 3. Model load/unload operations are themselves expressed as checkpoint-safe steps within the work-unit/round execution model (Section 7) — a round that was waiting on a model load can be safely interrupted and resumed exactly like any other round, with no special-casing required.

**4.5.4 Why not just LM Studio.** A single-user, GUI-driven model browser is a poor fit for unattended, multi-GPU, resource-elastic, many-concurrent-agent serving — it has no place in this design as the production runtime. It remains useful, narrowly, as the human workbench referenced in 4.5.2: a place to try a candidate model before it's admitted to the registry. The production serving path is the two-backend-plus-router architecture above, chosen specifically because it satisfies constraints this design already has (GPU-optional, elastic, checkpoint-safe, many-concurrent-caller) rather than because of any particular tool's popularity.

## 4.6 Sandboxed execution capability

Separately from model *inference*, the Engineering Master Agent (brain-design.md Section 2.2) needs to actually *run* code to verify claims — a different capability than serving an LLM's text output. The Body provides an isolated execution environment for this: - **Isolation:** no network egress, no access to any store beyond an explicitly provisioned scratch area, no persistence beyond the lifetime of a single execution request. - **Resource-bounded:** CPU/memory/time limits enforced per execution, scaled against currently available resources (Section 6) like any other work unit. - **Stateless per invocation:** an execution request receives code and inputs, returns output/pass-fail/error and nothing else — no execution's state carries into the next, mirroring the stateless-lease-holder pattern already used for distributed workers (4.2). -

**Structural trust boundary:** this sandbox's isolation guarantees are a hard dependency for the Brain's executable claim type (brain-design.md Section 3.5, 2.2) to mean what it claims to mean — this is flagged explicitly as a security-sensitive component warranting its own dedicated threat review before real code execution is enabled, not something to wave through as "just another work unit type."

---

## 5. Resident State Model (structural, not cognitive)

### 5.1 What the Body is responsible for holding

The Body owns the **storage, versioning, and durability** of these objects; it does not generate or interpret their contents: 1. **Belief Graph store** — a versioned graph store (nodes/edges/metadata schema only); population and interpretation of its contents is a Brain concern. 2. **Working-set cache** — scratch memory allocated to whatever processing units are active, managed for eviction under memory pressure. 3.

**Provenance Ledger store** — an index structure mapping claim IDs to source records; the Body guarantees it's queryable and durable, not that it's *correct*. 4. **Reputability Engine store** — versioned key-value/graph storage for grades and decision logs; the Body stores and serves this data, the grading logic itself is a Brain concern. 5. **Question Ledger store** — full lifecycle and version history for every submitted question; the Body guarantees permanence and retrievability.

### 5.2 Why all-in-memory

DDR3 is orders of magnitude faster than spinning disk and materially faster than most network storage. Given the expected access pattern (very large numbers of small, cross-referencing reads against a shared graph, from potentially many concurrent processing threads), keeping the working graph resident in DRAM is what makes sustained throughput possible at all. This is a structural justification, independent of what the graph's contents mean.

### 5.3 Persistence discipline

- **Checkpointing:** all five stores in 5.1 are checkpointed to Tier 2/3 on a configurable cadence, and always before a planned resource step-down or shutdown.
- **Append-only log:** every checkpoint is a new entry in an append-only history, never an overwrite. This is a storage-layer guarantee, independent of why the Brain wants history preserved.
- **Crash/eviction recovery:** on unclean shutdown, the Body reloads the last full checkpoint plus replayable log segments and resumes from the most recent safely-recoverable point. No stored object is ever dropped by a crash.

### 5.4 Memory pressure handling

1. Evict least-recently-used cache entries first (always re-fetchable from Tier 2/3).
  2. Page out the working-set context of the least-active queued work units (Section 7) to Tier 2/3, keeping only the most active ones fully resident.
  3. As a last resort, checkpoint-and-suspend rather than risk store corruption.
- 

## 6. Elastic Resource Management

### 6.1 Guiding rule

Anything except the DRAM allocation may be added or removed at any time. The Body must never lose data or progress because of this — only speed.

## 6.2 Resource monitor

Continuously tracks: available local/distributed cores, available GPU/VRAM, Tier 2 reachability and latency, remaining DRAM headroom, and current depth/mix of the work queue. Exposes this as a simple queryable state — reaction logic (6.3/6.4) consumes it but is implemented separately and testably against synthetic inputs.

## 6.3 Response to scale-down

- **Cores reduced:** scheduler reduces concurrently active work units; execution continues serially. Nothing queued is dropped, only deferred.
- **Distributed pool lost:** in-flight distributed work units are requeued or pulled back to local execution at reduced parallelism.
- **GPUs lost:** falls back to CPU-only paths; correctness unaffected, throughput reduced.
- **Tier 2 unreachable:** falls back to Tier 3 for reads/writes; checkpoint cadence automatically lengthens to avoid saturating HDD bandwidth (configurable bounds).
- **DRAM below floor:** triggers checkpoint-and-suspend, not a crash.

## 6.4 Response to scale-up

Newly available cores/GPUs/distributed nodes are picked up opportunistically at the next natural boundary in the execution model (Section 7) — not mid-unit, to avoid re-synchronization overhead — and used to widen parallel execution of the work queue.

---

# 7. Concurrency Execution Mechanics (structural only)

This section defines *how work is scheduled and executed*, not *what decisions drive prioritization or re-opening* — those are Brain concerns (importance rating, re-evaluation triggers). The Body only needs a generic, content-agnostic notion of a “work unit” with the following properties, supplied by whatever calls it:

- A unique ID and a position in a priority ordering (the ordering value itself is supplied externally; the Body just respects it).
- A sequence of discrete, checkpointable **rounds** — the Body does not know what a round *does*, only that it is atomic: fully complete or fully rolled back, with state persisted at each boundary.
- A status lifecycle: queued → active → suspended → completed → archived, with active → suspended → active being the pause/resume path used for time-slicing.

## 7.1 Single-unit runner

Executes one work unit’s rounds against pluggable round-handlers (stub or real), checkpointing after each round, and can be killed and resumed from the last completed round without data loss.

## 7.2 Multi-unit scheduler (time-sliced)

Extends 7.1 to a priority queue of many units, processing one round of one unit at a time when compute is constrained (down to the single-core floor), and yielding to the next queued unit after each round — this is pure round-robin/priority scheduling, with no logic about *why* one unit outranks another.

## 7.3 Multi-unit scheduler (parallel)

When resources allow, executes multiple units’ rounds genuinely concurrently, subject to a conflict-safe write path into the shared stores from Section 5.1. This requires: - A synchronization boundary at the end of each round (not mid-round) before writes are applied to shared stores. - A concurrency-control mechanism (e.g., optimistic locking with retry, or a single-writer queue in front of the shared store) to prevent lost updates. - A test harness that verifies a write from one unit’s round is visible to another unit’s next round.

## 7.4 Shared-store visibility

The Body’s job is limited to guaranteeing that writes to the stores in Section 5.1 are consistently visible across all active work units per the synchronization model in 7.3. It has no role in deciding what should be written or why one unit’s finding should affect another’s — that interpretation is entirely a Brain concern.

## 7.5 Concurrency-control mechanism: idempotency keys and expected-version writes

Section 7.3 requires “a concurrency-control mechanism” without specifying one; this fills that gap concretely, rather than leaving it to be improvised at implementation time: - **Expected-version writes:** every write to a shared store (Section 5.1) is submitted with the version/snapshot ID the caller last read. If the store’s current version no longer matches (another unit committed a write in between), the write is rejected and the caller is handed the new current state to retry against — an optimistic-concurrency pattern that prevents one unit’s update from silently clobbering another’s without either unit knowing. -

**Idempotency keys:** every write additionally carries a caller-supplied idempotency key (unique per logical write attempt). If a write is retried — because a work unit was interrupted and resumed (Section 4.4, 7.1) before confirming its previous write succeeded — the store recognizes the repeated key and returns the original result rather than applying the write twice. This is what makes checkpoint-and-resume (Section 5.4) and round-boundary retries (Section 7.1) safe by construction rather than by careful sequencing alone. - Together these two mechanisms are what Section 7.3’s “conflict-safe write path” concretely is: expected-version checks catch concurrent conflicts between different units, idempotency keys catch duplicate retries from the same unit — the two failure modes that “lost update” bugs in this kind of scheduler actually come from.

---

## 8. Configuration Surface (Body-relevant subset)

```
compute:
  local_cores_min: 1
  local_cores_max: 40
  distributed_cores_max: 200
  gpu_count_max: 10
  gpu_vram_pool_max_gb: 240

memory:
  local_dram_gb: 512          # treated as fixed/guaranteed
  distributed_dram_pool_max_gb: 1024
  working_set_floor_gb: <configurable, must be < local_dram_gb>

storage:
  boot_ssd_path: /mnt/boot-db
  nvme_lan_path: <network mount, optional>
  hdd_archive_path: /mnt/archive
  checkpoint_cadence_min_minutes: <config>
  checkpoint_cadence_max_minutes: <config>
  tier2_health_check_interval_seconds: <config>

concurrency:
  max_concurrent_active_units: <config, scales with available cores>
  time_slice_rounds_per_unit: <config>
  question_ledger_retention: permanent # hard invariant – never purge

ingestion:
  ingestion_enabled: true
  license_check_strict: true
  disallow_paid_apis: true           # hard invariant, not tunable

model_serving:
  gpu_backend: <engine identifier, e.g. vllm>
  cpu_fallback_backend: <engine identifier, e.g. llama.cpp>
  vram_pool_max_gb: 240             # mirrors compute.gpu_vram_pool_max_gb
  model_idle_unload_seconds: <config>
  gpu_wait_vs_cpu_fallback_threshold_seconds: <config> # how long to queue before redirecting to CPU
  registry_path: <content-addressed store location, Section 3.4>
  disallow_paid_model_apis: true     # hard invariant, not tunable – mirrors ingestion's no-paid-services

execution_sandbox:
  enabled: false                   # defaults off; enable only after dedicated threat review (Section 4)
  network_egress: false           # hard invariant, not tunable
  cpu_time_limit_seconds: <config>
  memory_limit_mb: <config>
  scratch_storage_max_mb: <config>

environment:
  bootstrap_mode: resume | genesis
  proxmox_node_id: <config>
  network_dependency_tolerance: strict | lenient
  degraded_mode_notifications: true
```

Fields governing reasoning depth, latency *targets*, importance weighting, and reputability standards belong to the Brain's configuration surface and are intentionally omitted here.

---

## 9. Ingestion Pipeline (mechanical only)

The Body is responsible for the **mechanics** of getting a source from the outside world into durable storage in a well-formed shape. It is not responsible for judging a source's reputability (a Brain concern).

1. **Fetch + access check:** given a source reference, fetch it and check license/terms-of-service/robots constraints. Output: accept/reject + reason. No paid or metered fetches are ever attempted — this is a hard invariant, not a policy the Body evaluates.
2. **Parse + normalize:** convert accepted raw sources into the Provenance Ledger's storage schema (Section 5.1).
3. **Scheduling hook:** ingestion runs as a background, resource-aware work unit type (Section 7), competing for resources like any other queued work — it does not get special priority by default.
4. **One-time seed load:** the initial foundational corpus load is a distinct, one-shot invocation of steps 1-2 against a fixed source list, kept separate from the general ongoing-ingestion code path.

What the Brain later does with ingested material — how it's graded, trusted, or reasoned over — is entirely out of scope here.

---

## 10. Resilience & Failure Modes Summary

| Failure/Change                                                          | Body Response                                                                                                                                                    |
|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Proxmox host reduces allocated cores to 1                               | Continue serially; time-slice across queued work units; no data loss                                                                                             |
| Distributed compute pool vanishes                                       | Requeue in-flight work; continue locally at reduced parallelism                                                                                                  |
| GPU pool vanishes                                                       | Fall back to CPU paths                                                                                                                                           |
| Tier 2 (NVMe/LAN) unreachable                                           | Fall back to Tier 3; widen checkpoint interval                                                                                                                   |
| Tier 3 (HDD) degraded (partial)                                         | Prioritize checkpoint writes; alert; continue read-mostly from Tier 2/DRAM                                                                                       |
| Unclean shutdown / VM eviction                                          | Resume from last checkpoint + replay log; Question Ledger and all stores fully intact                                                                            |
| DRAM allocation cut below working-set floor                             | Checkpoint-and-suspend all active work units; resume when restored                                                                                               |
| Ingestion target requires payment or violates ToS                       | Hard refusal; never attempted                                                                                                                                    |
| Work queue grows faster than compute can process it                     | Priority ordering (externally supplied) governs order; nothing dropped, only delayed                                                                             |
| Concurrent write conflict on shared store                               | Resolved via expected-version checks and idempotency keys (7.5); no silent lost update                                                                           |
| Checkpoint or source blob corrupted/tampered on disk                    | Content-address mismatch detected on read (3.4); falls back to prior good checkpoint in the hash chain                                                           |
| VRAM pool exhausted, GPU-backed model requested                         | Least-recently-used loaded model unloaded first; if still insufficient, request queued or redirected to CPU fallback (4.5.3)                                     |
| Model serving backend (GPU or CPU) crashes mid-request                  | Request retried against the alternate backend where possible; caller's round is unaffected beyond added latency, per the checkpoint-safe model-load step (4.5.3) |
| Model weight file corrupted/tampered on disk                            | Content-address mismatch detected on read (3.4, 4.5.1); model marked unavailable until re-fetched, never silently loaded                                         |
| Execution sandbox resource limit exceeded or isolation breach suspected | Execution terminated, result marked failed/untrusted, no claim of type executable produced from it (4.6)                                                         |

### 10.1 Recovery playbook pattern

Beyond the general failure-response table above, each row is additionally expressed as a runnable playbook: a named scenario, its detection signal, the exact recovery sequence, and the specific verification step that confirms recovery actually succeeded (not just that the response *ran*). This turns the table into something testable — each playbook is what Phase 10's fault-injection tasks (below) exercise directly — rather than

leaving “what recovery looks like in practice” implicit in prose.

---

## 11. Work Breakdown (Body-only phases, from the full task backlog)

These correspond to Phases 0, 1, 2, 3, 6, 7, 9, and 10 of the full system backlog — every task here can be built and fully tested against stub/no-op reasoning components.

**Phase 0 — Scaffolding & Contracts** 1. Repo structure, config schema + loader/validator. 2. Core data schemas (Belief Graph node/edge, Provenance entry, Question Ledger entry, Reputability grade) as standalone types with serialization tests. 3. Work-unit and round-handler interface definitions, with no-op implementations.

**Phase 1 — Storage Layer** 4. Tier 1 boot-load: read config from local path into memory. 5. Single-tier local key-value store for the schemas from task 2, with a test corpus. 6. Tiered read/write: Tier 2 preferred, Tier 3 fallback, with a mocked “Tier 2 unreachable” harness. 7. Append-only checkpoint writer + snapshot ID generation for all five stores in Section 5.1. 7a. Content-addressed storage layer (3.4): hash-based keys, tamper detection on read, and hash-chained checkpoint log linking.

**Phase 2 — Resource Monitor & Elasticity** 8. Resource monitor: polls cores/GPUs/memory/Tier 2 reachability, exposes state via API. 9. Scale-down reaction rules as pure functions over synthetic resource-state inputs. 10. Checkpoint-and-suspend triggered by simulated DRAM-floor breach; verify identical state reconstruction on resume.

**Phase 3 — Question Ledger & Single-Unit Scheduler** 11. Question Ledger CRUD (submit/get/list-by-status) against the storage layer. 12. Single-work-unit runner (7.1) executing against no-op round-handlers. 13. Round-boundary checkpointing for the single-unit runner; verify kill-and-resume continues from the last completed round.

**Phase 6 — Ingestion Pipeline (mechanical)** 14. Fetch + license/ToS check module, isolated, input URL → accept/reject + reason. 15. Parse/normalize accepted sources into Provenance Ledger schema. 16. Wire ingestion as a scheduled background work-unit type (Section 7). 17. One-time foundational corpus seed-load task, separate from general ingestion code.

**Phase 7 — Concurrency** 18. Extend the scheduler from one work unit to a time-sliced priority queue (7.2) — highest-risk task in this document; build and review in isolation. 19. Cross-unit visibility test: a write from unit A’s round is visible in unit B’s next round. 20. True parallel execution (7.3) gated by the resource monitor, with a conflict-safe write path and concurrency-control mechanism. 20a. Expected-version write checks and idempotency-key deduplication (7.5), tested against simulated concurrent-write and duplicate-retry scenarios.

**Phase 9 — Acceleration & Distributed Compute** 21. CPU-only baseline for embedding/retrieval-style workloads, tested in isolation. 22. GPU-accelerated path behind a feature flag; automated test that both paths agree on output. 23. Distributed worker dispatch for one already-parallel workload, with requeue-on-loss tested via simulated worker failure.

**Phase 9b — Local Model Serving Layer (new)** 23a. Model registry schema and content-addressed weight storage (4.5.1), integrated with the Phase 1 content-addressing layer (task 7a). 23b. GPU-backed serving integration (single model, single backend) behind the unified routing interface (4.5.2). 23c. CPU-backed fallback serving integration behind the same routing interface; automated test that both paths return usably-equivalent responses for the same request. 23d. VRAM budget management: least-recently-used unload, queue-or-redirect-to-CPU logic (4.5.3), tested against synthetic concurrent-demand scenarios. 23e. Checkpoint-safe model-load step wired into the work-unit/round execution model (Section 7), verified killable/resumable mid-load like any other round. 23f. Model workbench admission hook: a manual/human step that registers a newly evaluated model into the registry (4.5.2, 4.5.4) — deliberately separate from the query-time serving path.

**Phase 9c — Sandboxed Execution Capability (new, security-sensitive)** 23g. Isolation boundary implementation: no network egress, no persistent state beyond scratch, resource limits enforced (4.6) — build and review this task in isolation, with a dedicated security review before enabling by default (`execution_sandbox.enabled`, Section 8). 23h. Stateless per-invocation execution interface: code + inputs in, output/pass-fail/error out, nothing else. 23i. Fault-injection scenarios specific to the sandbox: resource-limit breach, attempted network access, attempted persistence — each verified to fail closed, not silently succeed.

**Phase 10 — Integration & Fault Injection** 24. End-to-end test: boot cold, seed corpus, submit N concurrent work units (stub round-handlers), verify all reach completed with correct checkpoint history. 25. Fault-injection suite, written as the recovery playbooks from Section 10.1: kill Tier 2, drop to 1 core, cut DRAM toward the floor, kill mid-round, corrupt a checkpoint blob on disk — one playbook per task, each

verifying not just that the response ran but that the defined verification step confirms recovery. 26. Long-run soak test: idle scheduler running against a static queue for an extended period, verify no store corruption or resource-monitor drift.

---

*End of Body design document. This document is self-contained and buildable in full against stub/no-op reasoning components. The Brain (Master Agents, deliberation content, Reputability Engine judgment, and re-evaluation triggers) is covered separately and is out of scope here until that work resumes.*